PROYECTO COMPILADORES

Implementación de un Analizador Sintáctico para LP

Dos entregables: 30% + 70%

1. Alcance y reglas de implementación

3.1. Flujo general

```
Archivo fuente (.lp) → Analizador léxico → Secuencia de tokens → Analizador sintáctico → Resultado / AST / errores
```

3.2. Requisitos técnicos

- Reutilizar el analizador léxico desarrollado para LP. No se debe volver a analizar el código fuente carácter por carácter desde el módulo sintáctico.
- El analizador sintáctico debe consumir la secuencia de tokens producida por el lexer.
- Implementar el Analizador Sintactico mediante descenso recursivo: las funciones o métodos deben corresponder, de manera comprensible, a los no terminales de la gramática.
- Distinguir entre aceptación del programa y detección de errores sintácticos.
- Mantener separado el análisis sintáctico del análisis semántico. En este proyecto no se exige comprobar declaración de variables, compatibilidad de tipos ni ejecución del programa.
- El código debe estar modularizado, comentado razonablemente y organizado para facilitar su revisión.

3.3. Convenciones de tokens

Los terminales de la gramática deben asociarse con los tipos de token emitidos por el analizador léxico. Por ejemplo, id corresponde al token de identificador; num_int y num_dec corresponden a constantes numéricas; texto corresponde a cadenas de texto; comp representa cualquiera de los operadores relacionales definidos por el lexer.

Antes de la implementación completa, cada equipo debe verificar que los nombres de tokens del lexer coincidan con los terminales de la gramática. La producción de llamada a función utiliza el terminal func, que no aparece en la lista léxica inicial proporcionada; el equipo debe incorporarlo al lexer como palabra reservada.

3.4. Consideraciones importantes de la gramática

- La estructura principal es PROGRAMA → LISTA_FUNCIONES PRINCIPAL: las declaraciones de funciones preceden a main.
- La producción PRINCIPAL requiere main() seguido de un BLOQUE.
- Un BLOQUE tiene la forma { LISTA_VARIABLES COMANDOS }; por tanto, las declaraciones de variables aparecen antes de los comandos dentro del bloque.
- La gramática incluye selección if/else y repetición while. No incluye una producción para for; por ello, for no forma parte de los requisitos de este proyecto.

- La gramática define expresiones con niveles de precedencia: operadores multiplicativos, aditivos, relacionales, && y ||.

4. Entregable 1 – Implementación inicial (30%)

Fecha de entrega: 13 de Noviembre.

Propósito: demostrar que el equipo cuenta con la integración funcional entre lexer (Analizador Léxico) y parser (Analizador Sintáctico) y que puede reconocer la estructura básica de un programa LP, incluyendo bloques, declaraciones y asignaciones sencillas.

4.1. Funcionalidades obligatorias

- Integración con el analizador léxico: leer un archivo .lp y obtener su secuencia de tokens.
- Reconocimiento de PROGRAMA y PRINCIPAL: aceptar la estructura general de un programa que contiene main() y su bloque.
- Reconocimiento de BLOQUE: validar llaves de apertura y cierre.
- Reconocimiento de declaraciones: implementar TIPO, TIPO_BASE, DIMENSION, LISTA_VARIABLES y LISTA_ID, según corresponda a las construcciones incluidas en esta etapa.
- Reconocimiento de asignaciones básicas: implementar ASIGNACION y COMPLEMENTO para asignaciones con expresiones sencillas.
- Detección de errores sintácticos básicos, como punto y coma faltante, paréntesis o llaves no cerradas y token inesperado.

4.2. Ejemplo mínimo que debe reconocer

```
void main() {
    int edad;
    float promedio;
    edad = 20;
    promedio = 15.5;
}
```

4.3. Ejemplo que debe reportar como incorrecto

```
void main() {
    int edad
    edad = 20;
}
```

En el ejemplo incorrecto falta el punto y coma al finalizar la declaración de la variable. El parser debe detectar el problema y presentar un mensaje comprensible.

4.4. Evidencias de la primera entrega

- Código fuente del proyecto, incluyendo la integración con el lexer.
- Archivo(s) .lp utilizados para las pruebas.
- Evidencia de ejecución con al menos un programa válido y dos casos inválidos.
- Breve documento README con instrucciones para compilar y ejecutar el sistema.
- Sustentación breve: cada integrante debe explicar una parte del flujo y responder preguntas sobre el código.

5. Entregable 2 – Implementación completa (70%)

Fecha de entrega: 24 de Noviembre .

Propósito: completar el analizador sintáctico para cubrir las construcciones restantes de la gramática, mejorar el manejo de errores y generar una representación estructurada del programa.

5.1. Funcionalidades obligatorias

- Expresiones: implementar la jerarquía completa de expresiones y respetar la precedencia y asociatividad definida por las producciones de la gramática.
- Factores y términos: reconocer identificadores, constantes, texto, signos unarios, negación lógica y expresiones entre paréntesis.
- Entrada y salida: reconocer scanf y println, incluyendo sus listas de términos.
- Estructuras de control: reconocer if, else y while, con sus condiciones y bloques.
- Funciones: reconocer declaraciones de funciones, tipos de retorno, parámetros, llamadas y argumentos, conforme a la gramática.
- Retorno: reconocer return seguido de una expresión y punto y coma.
- AST: construir una representación estructurada del programa. Puede utilizarse una jerarquía de nodos o una estructura equivalente.
- Manejo de errores: emitir mensajes que indiquen la ubicación aproximada del error, el token encontrado y el token o construcción esperada. Cuando sea viable, continuar el análisis para reportar más de un error.
- Integración final: ejecutar el flujo completo desde el archivo .lp hasta el resultado del análisis sintáctico y la generación/visualización del AST.

5.2. Ejemplo de programa para pruebas integrales

```
void main() {
    int edad;
    float promedio;
    edad = 20;
    promedio = 15.5;

    if (edad >= 18 && edad <= 60) {
        println("Edad valida");
    } else {
        println("Edad no valida");
    }

    while (edad > 0) {
        edad = edad - 1;
    }

    return edad;
}
```

Este ejemplo ilustra varias construcciones incluidas en la gramática. Los equipos deben complementar las pruebas con casos que incluyan funciones, parámetros, llamadas, lectura, arreglos y expresiones con distintos operadores.

5.3. Evidencias de la segunda entrega

- Código fuente completo y actualizado en el repositorio GitHub del equipo.
- README con requisitos, instrucciones de compilación/ejecución, organización de módulos y ejemplos.
- Conjunto de archivos .lp de prueba: válidos e inválidos, cubriendo las principales producciones de la gramática.
- Salida del análisis: aceptación/rechazo, mensajes de error y representación del AST.

- Breve informe técnico que describa la arquitectura, decisiones de implementación, limitaciones conocidas y distribución del trabajo.
- Sustentación y demostración en vivo.

8. Organización del equipo y sustentación

- El trabajo puede distribuirse por módulos, pero la integración debe ser responsabilidad de todo el equipo.
- Todos los integrantes deben conocer el flujo completo: archivo fuente, lexer, tokens, parser, AST y reporte de errores.
- Durante la sustentación, cualquier integrante podrá ser consultado sobre cualquier módulo.
- La evaluación considera tanto el funcionamiento grupal como la comprensión individual demostrada durante la explicación y la demostración.

9. Formato de entrega

- Repositorio GitHub del equipo con el código fuente actualizado.
- Carpeta de pruebas con archivos .lp y, cuando corresponda, las salidas obtenidas.
- README.md con instrucciones claras para compilar y ejecutar.
- Informe técnico breve para la entrega final.
- El repositorio debe permitir identificar la contribución de los integrantes mediante commits, módulos o una descripción de responsabilidades.